

6-1강. 마이크로서비스 설계 상세

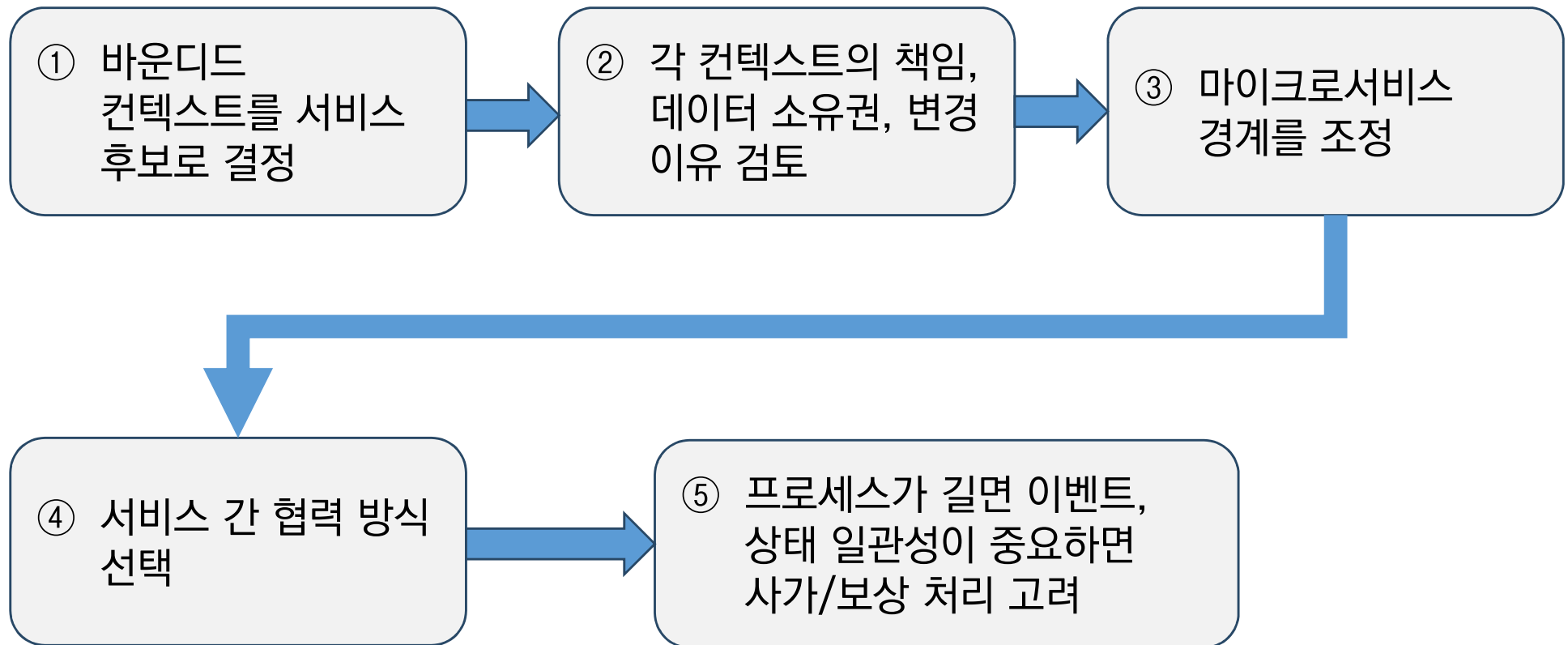


한국공학대학교
TECH UNIVERSITY OF KOREA

이영곤

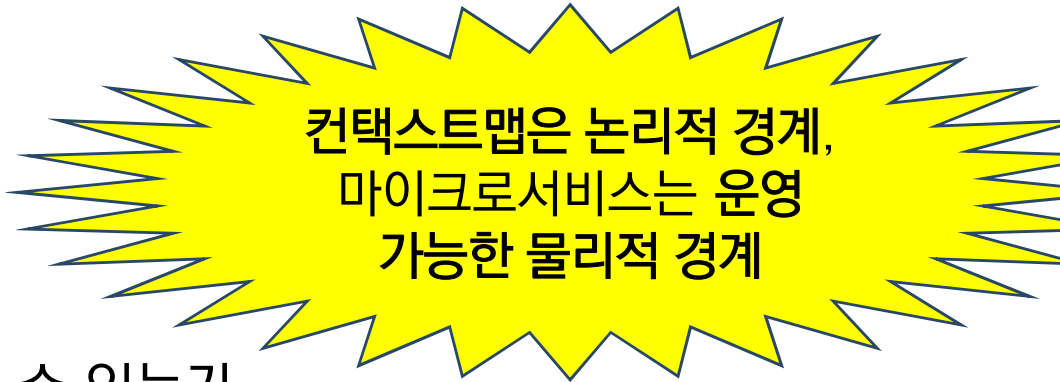


조정 프로세스



컨택스트맵에서 마이크로서비스 도출

- ✓ 1 컨텍스트 ≠ 1 서비스
- ✓ 서비스 분리 기준
 - 업무 책임이 독립적인가
 - 자기 데이터베이스를 독립적으로 가질 수 있는가
 - 변경 주기가 다른가
 - 배포를 따로 해야 하는가
 - 트래픽/성능 특성이 다른가
 - 조직(팀) 단위로 ownership을 나눌 수 있는가
- ✓ 예 :
 - 상품관리 컨텍스트 → Product Service
 - 주문 컨텍스트 → Order Service



컨택스트맵은 논리적 경계,
마이크로서비스는 운영
가능한 물리적 경계

서비스간 협력 방식 결정(동기식)

✓ 사용 기술

- REST, gRPC, Feign Client, ...

✓ 특징:

- 바로 응답이 필요할 때 적합
- 구현이 단순하고 이해하기 쉬우나 서비스 간 결합도 상승
- 장애 전파, 타임아웃, 연쇄 실패 문제가 생김

✓ 적합한 경우:

- 화면 조회용
- 즉시 결과가 필요한 단건 처리
- 강한 실시간 응답 요구

활용 사례

- Order Service가 Product Service에 상품 정보 조회

서비스간 협력 방식 결정(비동기식)

✓ 사용 기술:

- Kafka, RabbitMQ, ActiveMQ, NATS, WebClient,...

✓ 특징:

- 발신자와 수신자가 시간적으로 분리됨
- 느슨한 결합
- 확장성과 복원력이 좋음
- 추적, 중복 처리, 순서 보장, 재처리 설계가 필요

✓ 적합한 경우:

- 후속 처리서비스 간 상태 전파
- 이벤트 기반 협업대량 처리

활용 사례

- OrderCreated 이벤트 발행
- Inventory, Payment, Delivery가 이를 구독

서비스간 협력 방식 결정(데이터 통합/복제 기반 협력)

✓ 개념:

- 서비스 간 직접 호출을 줄이고, 데이터 자체를 “보내서” 정합성과 확장성을 확보하는 방법
- 데이터를 복제/전파해서 서비스 각자 자기 데이터로 처리
- API 호출 기반 협력 → 데이터 기반 협력

✓ 목적:

- 결합도 감소 (decoupling)
- 비동기 처리 (eventual consistency)
- 읽기 성능 개선 (local read model)
- 장애 격리

서비스간 협력 방식 결정(데이터 통합/복제 기반 협력)

✓ 사용 기술:

- CDC(Change Data Capture)
- Outbox Pattern
- ETL/Batch
- Read Model 복제
- 이벤트 소싱 일부 구조

✓ 특징:

- 서비스 DB 직접 조회를 피하고
- 필요한 데이터만 다른 서비스나 조회 모델로 복제
- 분산 조인 문제를 줄일 수 있음

활용 사례

- Product 변경 내용을 이벤트로 발행해서 Search/Read Model에 반영
- Order DB 변경을 CDC로 Kafka에 보냄

데이터 통합/복제 기반 협력: CDC(Change Data Capture)

✓ 개념

- DB에서 변경된 데이터만 추출해서 이벤트로 전파

✓ 특징

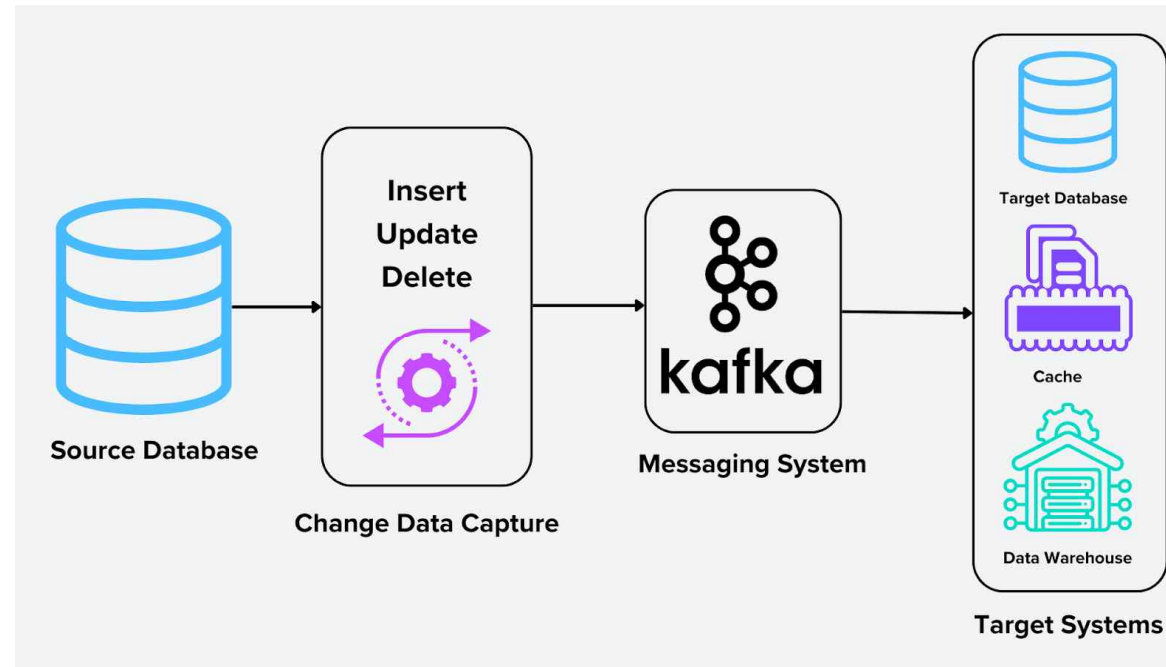
- 코드 수정 없이 데이터 흐름 생성
- DB가 “이벤트 소스” 역할

✓ 장점

- 기존 시스템 쉽게 MSA화 가능
- 데이터 유실 거의 없음

✓ 단점

- 비즈니스 의미 없는 “저수준 이벤트”
- 테이블 구조에 강하게 의존



데이터 통합/복제 기반 협력: Outbox Pattern

✓ 개념

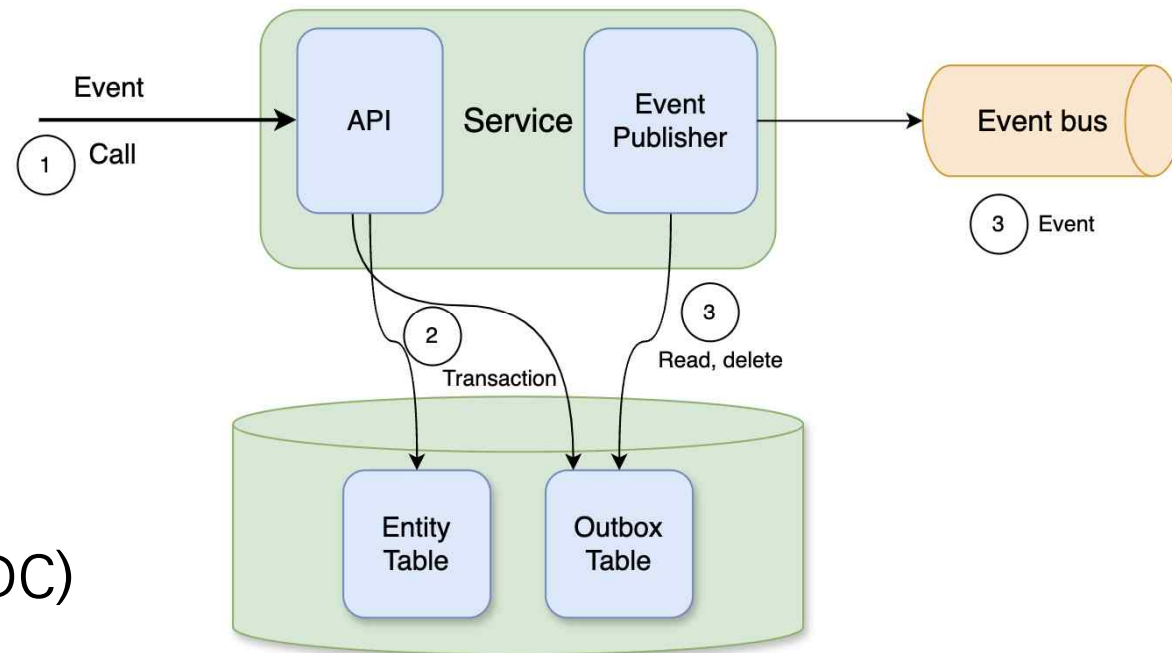
- 서비스가 트랜잭션 내에서: DB + 이벤트를 동시에 기록
- DB 트랜잭션과 이벤트 발행을 원자적으로 처리

✓ 장점

- 이벤트 유실 방지
- 데이터와 이벤트 정합성 보장

✓ 단점

- outbox 테이블 관리 필요
- 추가 컴포넌트 필요 (poller or CDC)



데이터 통합/복제 기반 협력: ETL / Batch

✓ 개념

- 주기적으로 데이터를 추출해서 변환 후 적재

✓ 특징

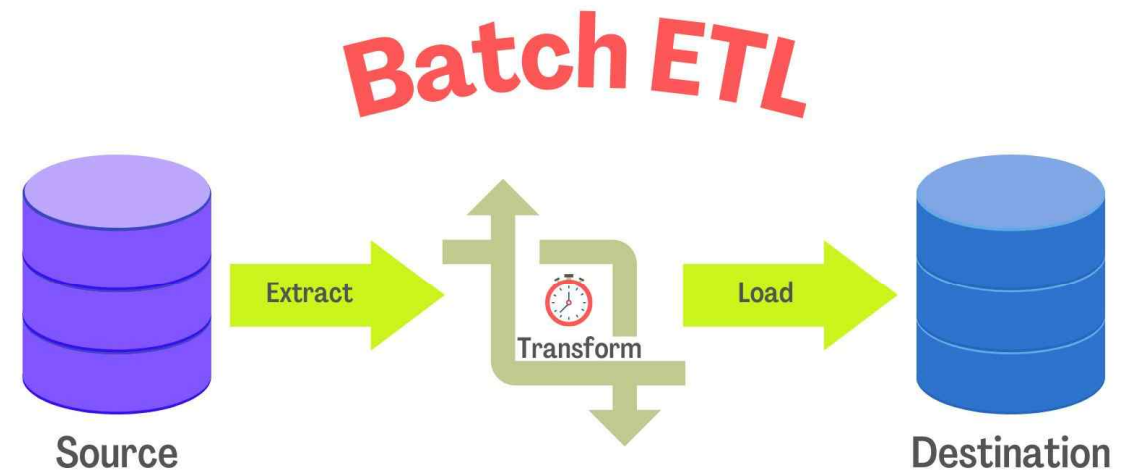
- 실시간 아님 (batch)
- 대량 데이터 처리에 강함

✓ 장점

- 대규모 데이터 분석
- 구조 변환 가능

✓ 단점

- 지연(latency)
- 실시간 서비스에는 부적합



데이터 통합/복제 기반 협력: CQRS(Command Query Responsibility Separation)

✓ 개념

- 쓰기 모델과 읽기 모델을 분리

✓ 특징

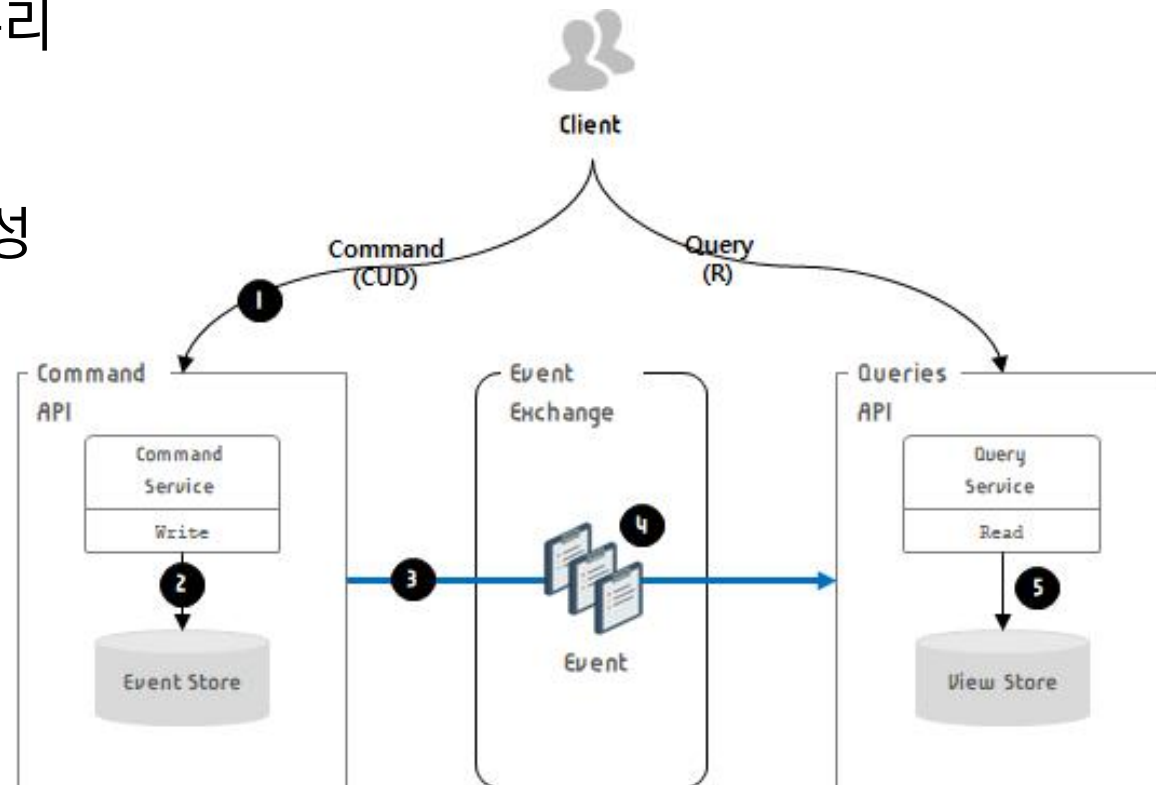
- 읽기 최적화된 DB 별도 구성

✓ 장점

- 조회 성능 극대화
- 서비스 간 DB 호출 제거

✓ 단점

- 데이터 일관성 지연
- 설계 복잡



데이터 통합/복제 방식과 이벤트 방식의 차이

구분	이벤트 처리 방식	데이터 통합/복제 방식
정의	서비스 간 비동기 이벤트로 협업	데이터를 여러 서비스에 복제/전파하여 각자 보유
관점	통신(Communication)	데이터(Data)
목적	후속 작업 트리거 (행위 중심)	조회/분석/독립성 확보 (데이터 중심)
핵심 질문	“누가 다음 일을 할 것인가?”	“누가 이 데이터를 가지고 있어야 하는가?”
데이터 역할	처리에 필요한 입력값	각 서비스의 로컬 데이터 구성 요소
처리 방식	이벤트 수신 → 로직 실행 → 끝	이벤트/CDC → DB 반영 → 지속적 유지
상태 저장	필요 없음 (stateless 가능)	반드시 필요 (read model, projection)
결합도	낮음 (이벤트 기반)	더 낮음 (데이터 독립성 확보)

이벤트

- ✓ 이벤트 발행/구독 (Pub/Sub)
 - 어떤 서비스가 상태 변화를 이벤트로 발행
 - 여러 서비스가 구독해서 각자 반응
 - 예: OrderCreated, PaymentCompleted, InventoryReserved, DeliveryStarted
- ✓ 장점:
 - 느슨한 결합
 - 발행자와 소비자가 완전 독립적
- ✓ 주의:
 - 명령과 이벤트는 다르게 작성해야 함

커맨드 메시지

- ✓ 다른 서비스에 주어지는 명령과 같음
 - 예: ReserveInventory, ProcessPayment, CreateShipment
- ✓ 특징:
 - 대상 서비스가 비교적 명확
 - 오케스트레이션에서 자주 사용
- ✓ 장점:
 - 흐름이 명확하고 프로세스 처리에 유용
- ✓ 주의:
 - 결합도가 이벤트보다 높음
 - 중앙 제어가 강해질 수 있음

이벤트 스트리밍

- ✓ Kafka 계열에서 많이 사용
- ✓ 특징:
 - 이벤트가 로그처럼 계속 쌓임
 - 소비자는 지금까지 처리한 시점(Offset)부터 읽거나 재처리 가능
 - 실시간 분석, 감사 추적, 상태 재구성에 유리
- ✓ 예:
 - 주문/결제/배송 이벤트를 스트림으로 적재
 - 조회 모델 생성
 - 모니터링/감사 로그 활용

CDC + Outbox

✓ 문제:

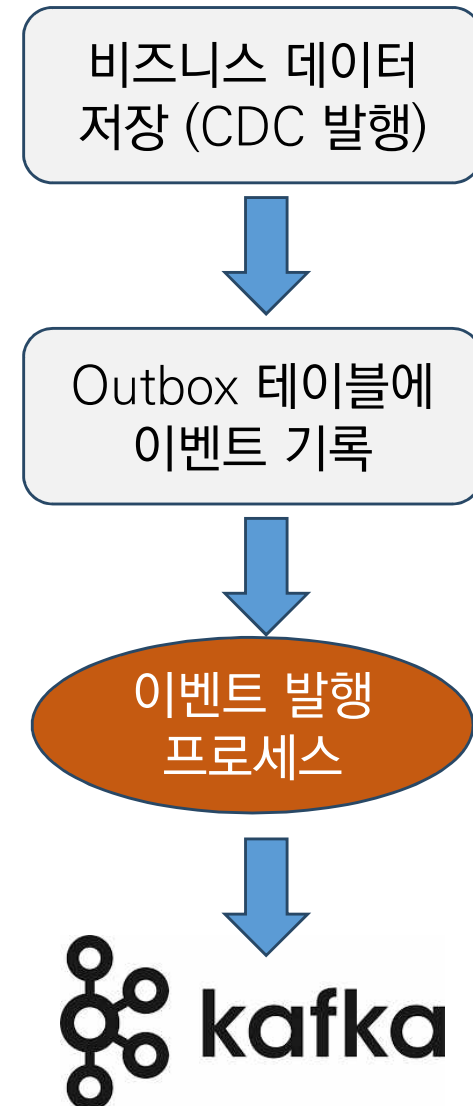
- DB 저장은 됐는데 이벤트 발행이 실패하면 데이터와 메시지가 불일치

✓ 해결:

- 비즈니스 데이터 저장과 함께 Outbox 테이블에 이벤트 기록
- 이후 별도 프로세스가 Outbox를 읽어 브로커(Kafka 등)로 발행

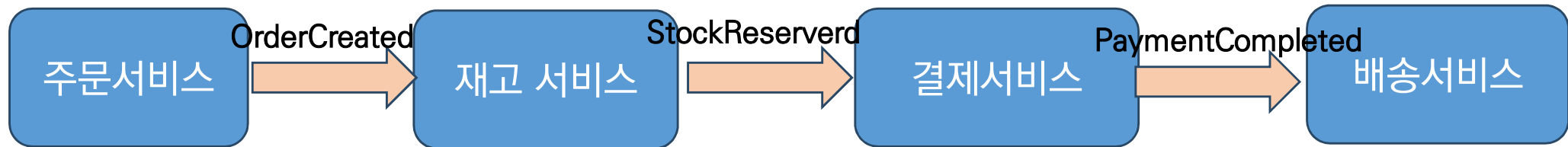
✓ 장점:

- DB 상태와 이벤트 발행 간 정합성 향상
- “이벤트 유실” 문제를 줄임



Choreography 방식

- ✓ 각 서비스가 이벤트를 듣고 스스로 다음 행동 결정



- ✓ 장점:

- 중앙 제어기 없음
- 느슨한 결합
- 확장에 유리

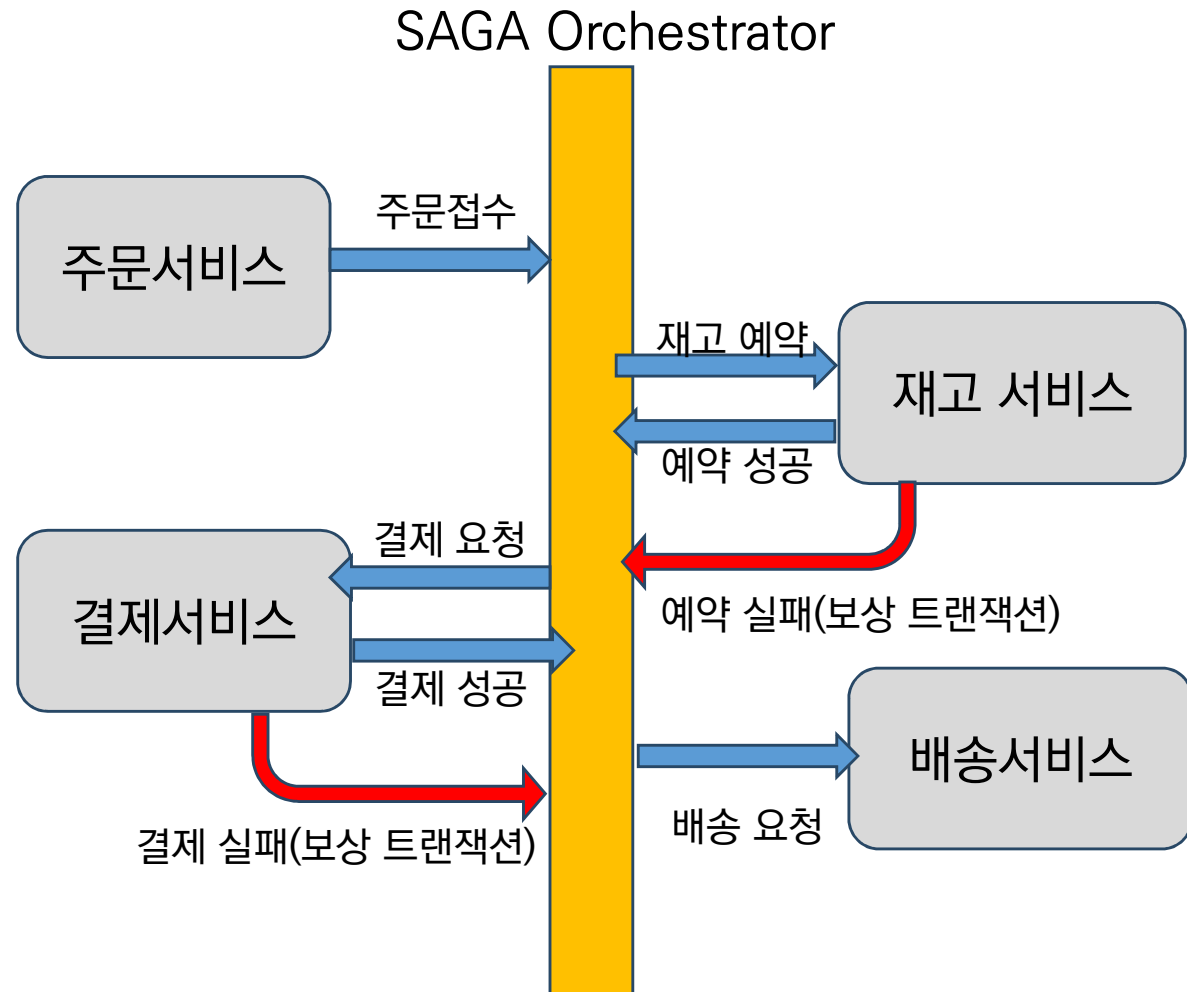
- ✓ 단점:

- 전체 흐름이 눈에 잘 안 보임
- 복잡해지면 추적이 어려움
- 어디서 실패했는지 파악이 어려움

비교적 단순한
프로세스
이벤트 중심 처리

Orchestration 방식

- ✓ 오케스트레이터가 흐름을 지휘
- ✓ 장점:
 - 프로세스 흐름이 명확
 - 예외 처리와 상태 관리가 쉬움
- ✓ 단점:
 - 중앙 서비스가 복잡해짐
 - 오케스트레이터에 의존성이 생김
- ✓ 적합:
 - 절차가 명확한 업무
 - 실패/보상 로직이 중요한 경우



이벤트/메시징 방식 선택 기준

- ✓ 조회는 동기, 상태 변경은 비동기
 - 상품 상세 조회 → REST
 - 주문 생성 후 후속 처리 → Kafka 이벤트
 - 재고/결제/배송 후속 반영 → 비동기
- ✓ 명령은 동기, 결과 전파는 비동기
- ✓ 내부 핵심 프로세스는 Saga, 부가 기능은 이벤트 기반
 - 예:주문-결제-재고는 Saga알림, 로그, 통계, 추천은 이벤트 구독 처리

메시징 설계시 고려사항

- ✓ 메시지 종류 정의
 - 이벤트명
 - 커맨드명
 - payload 구조
 - 필수/선택 필드
 - 버전 관리 방식
 - 예: OrderCreated, PaymentFailed, InventoryReserved
- ✓ 전달 보장 수준
 - at most once, at least once, exactly once
 - 실무에서는 대부분 at least once를 기준으로 함
 - 소비자 쪽에서 중복 처리 방지(idempotency)를 두어야 함

메시징 설계시 고려사항

✓ 순서 보장

- 같은 주문 ID 기준으로는 순서를 보장할지
- 전체 순서는 포기할지
- 파티션 키를 무엇으로 할지
- Kafka에서는 보통 orderId를 key로 써서 같은 주문 관련 이벤트 순서를 맞춤

✓ 실패 처리

- 재시도
- DLQ(Dead Letter Queue): 메인 큐 → 처리 실패 → DLQ 이동
- 타임아웃
- 보상 트랜잭션
- 수동 복구 절차

메시징 설계시 고려사항

- ✓ 데이터 정합성
 - 즉시 일관성
 - 최종 일관성
 - 사용자 화면에서 비일관성 허용 여부. 예: 주문 직후 잠깐 재고 화면이 늦게 반영될 수 있음
- ✓ 관측성
 - 분산 추적(Tracing): Correlation ID를 활용
 - 비즈니스 트랜잭션 추적: Saga ID 활용
 - 로그 상관관계

Customer-Supplier

- ✓ Supplier가 업스트림, Customer가 다운스트림
- ✓ Supplier가 어떤 기능/데이터를 제공하고 Customer가 그것을 사용
- ✓ Customer 요구를 Supplier가 어느 정도 반영해 주는 협력 관계
- ✓ 메시징 관점의 룰
 - 이벤트 계약은 Supplier가 주도하지만 Customer 요구를 반영
 - 이벤트 이름과 payload는 합의된 Published Language처럼 관리
 - 하위 호환성(backward compatibility)을 지킴
 - 스키마 버전 관리가 중요
 - 변경 시 Supplier가 일방적으로 계약을 깨면 안됨

Conformist

- ✓ 다운스트림이 업스트림에 모델을 맞추는 관계
- ✓ 메시징 관점의 룰
 - 업스트림 메시지 형식을 그대로 수용
 - 다운스트림은 자체 의미 체계를 주장하지 않음
 - 변환을 최소화해야 함
 - 업스트림 변경에 취약하므로 결합도가 높음
 - 내부 모델이 오염될 가능성을 감수해야 함
- ✓ 보통 쓰는 방식
 - 업스트림 event schema 그대로 소비
 - 필드명, 상태값, enum도 그대로 따라감
 - 단순 어댑터 수준의 매핑만 수행

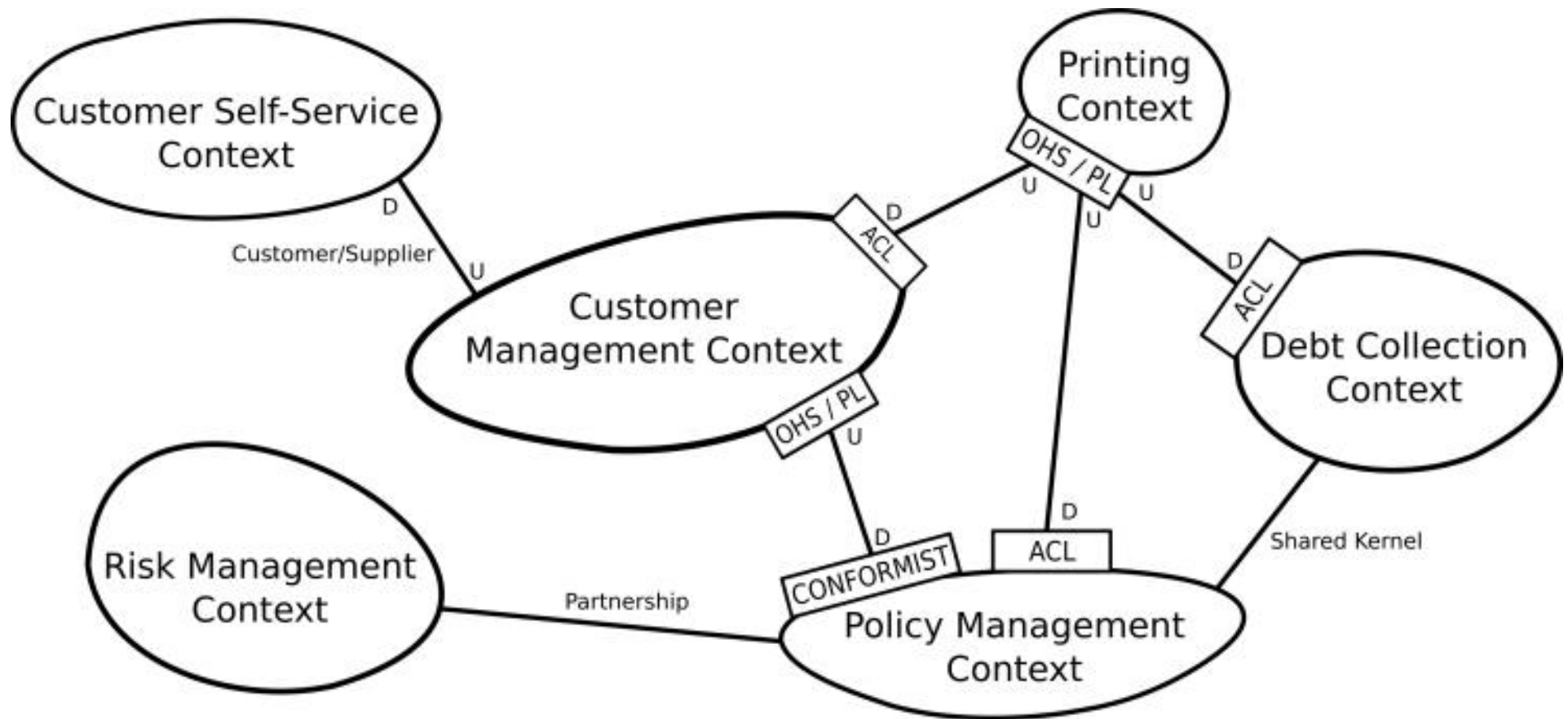
Anti-Corruption Layer(ACL)

- ✓ 외부 모델이 모델을 오염시키지 못하게 번역층을 둠
- ✓ 메시징 관점의 룰
 - 외부 이벤트/메시지를 내부 도메인 이벤트로 변환
 - 변환 책임은 ACL 계층에 둠
 - 외부 스키마와 내부 스키마를 분리
 - 업스트림 변경의 충격을 ACL에서 흡수
- ✓ 보통 쓰는 방식
 - Consumer → Translator → Internal Event
 - External topic 구독 후 internal topic 재발행
 - Canonical model 또는 local domain model 변환
 - Adapter + Mapper + Translator 패턴

Shared Kernel

- ✓ 두 컨텍스트가 일부 모델을 명시적으로 공유하는 관계
- ✓ 메시징 관점의 룰
 - 공유되는 개념은 공통 스키마/공통 라이브러리로 관리
 - 공유 범위는 매우 작게 유지해야 함
 - 공유 모델 변경은 공동 거버넌스가 필요
 - 공통 Value Object나 식별자 규칙 정도만 공유
- ✓ 보통 쓰는 방식
 - 공통 이벤트 스키마 정의
 - 공통 ID 타입, Money, Address 같은 value object 공유
 - shared library 또는 schema repository

컨택스트맵 사례



- D: Downstream
- U: Upstream
- OHS: Open Host Service ⇒ 공개인터페이스, API
- PL: Published Language ⇒ JSON/schema/event 구조

유형별 메시징 정리

관계 유형	메시지 형식 주도권	변환 책임	결합도	권장 방식
Customer-Supplier	Supplier 중심, Customer 협의 가능	양측 협의, 보통 최소 변환	중간	계약 기반 이벤트, 버전 관리
Conformist	Upstream 절대 우위	Downstream이 흡수	높음	그대로 수용
ACL	내부 컨텍스트 중심	ACL 계층	낮음	번역 후 내부 이벤트 재발행
Shared Kernel	공동 관리	최소 변환	중간~높음	공통 스키마 일부 공유
OHS/PL	Supplier가 표준 인터페이스 제공	Downstream에서 사용	중간	그대로 수용

데이터 복제에 따른 문제

- ✓ 원칙적으로는 복제를 피해야 함
 - 데이터 중복 → 불일치 위험
 - 수정 시 여러 곳 동기화 필요
 - 트랜잭션 관리 어려움
- ☞ 서비스는 자기 데이터만 가진다



이 원칙만 고집하면 문제 발생

데이터를 복제하지 않음에 따른 문제

- ✓ 문제 1: 서비스콜 체인 발생시 조회 성능 붕괴
 - 예: 화면 → Order → Inventory → Pricing → Delivery
 - ☞ 서비스 4~5개 호출시 조회시간이 너무 길어짐
- ✓ 문제 2: 강한 결합에 따른 변경 영향과 장애 전파
 - 예: Inventory API 바뀌면 Order 영향
 - 장애 전파

데이터를 복제하지 않음에 따른 문제

- ✓ 문제 3: 비즈니스 불가능
 - 예: 현재 재고 기준으로 주문 가능 여부 판단
 - 예: 고객별 구매 패턴 분석
- ☞ 다른 서비스 데이터 없으면 못함



필요한 데이터만 복제해서 로컬에서 처리

데이터 복제에 따른 효과

✓ 서비스 독립성 확보

- 복제 없으면: Order → Inventory API 호출
- 복제 있으면: Order → 자기 DB에서 재고 조회

↳ 다른 서비스 의존 제거

✓ 조회 성능 개선

- 여러 서비스 호출 → 1번 조회로 끝

↳ Read Model / CQRS 핵심

데이터 복제에 따른 효과

✓ 장애 격리

- 복제 없으면: Inventory 죽으면 → 주문도 영향
- 복제 있으면: 최신은 아니어도 → 서비스 계속 동작

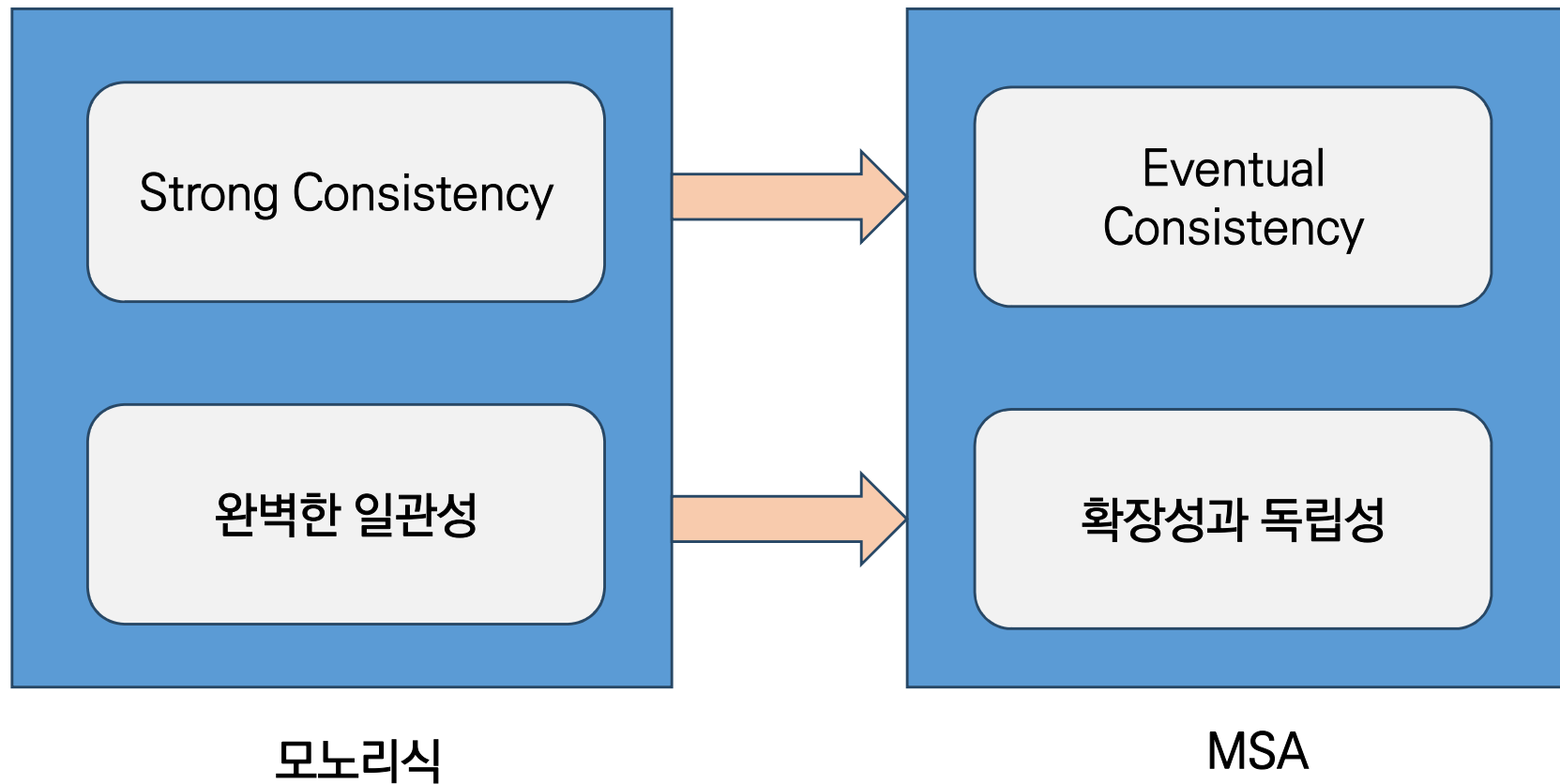
☞ availability 증가

✓ 데이터 활용 (분석/AI)

- 빅데이터 분석이나 AI 학습용 데이터 구축

☞ 복제 없이는 불가능

데이터 복제 트레이드오프



데이터 복제 규칙

- ✓ 원본(Source of Truth)”은 하나만
 - 예: 주문 → Order 서비스
 - 예: 재고 → Inventory 서비스
- ☞ 복제본은 읽기용
- ✓ 복제는 “필요한 최소 데이터만” 해야 함
 - 전체 테이블 복제 안됨
 - 필요한 필드만 복제

데이터 복제 규칙

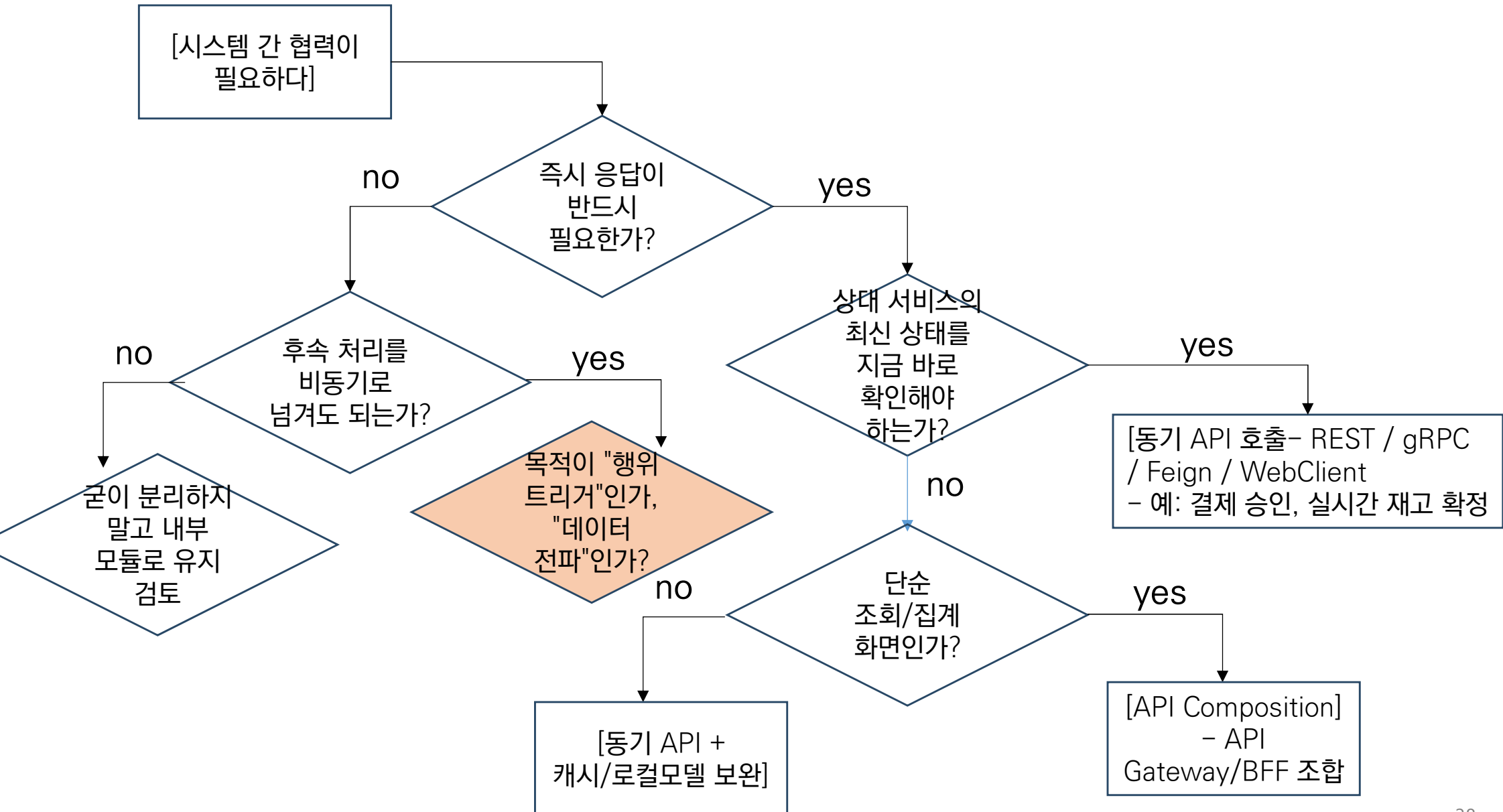
- ✓ 이벤트 기반 동기화 필수
 - 변경 발생 → 이벤트 → 다른 서비스 반영
- ✓ 일관성 지연을 허용
 - 몇 ms ~ 몇 초 늦어도 OK

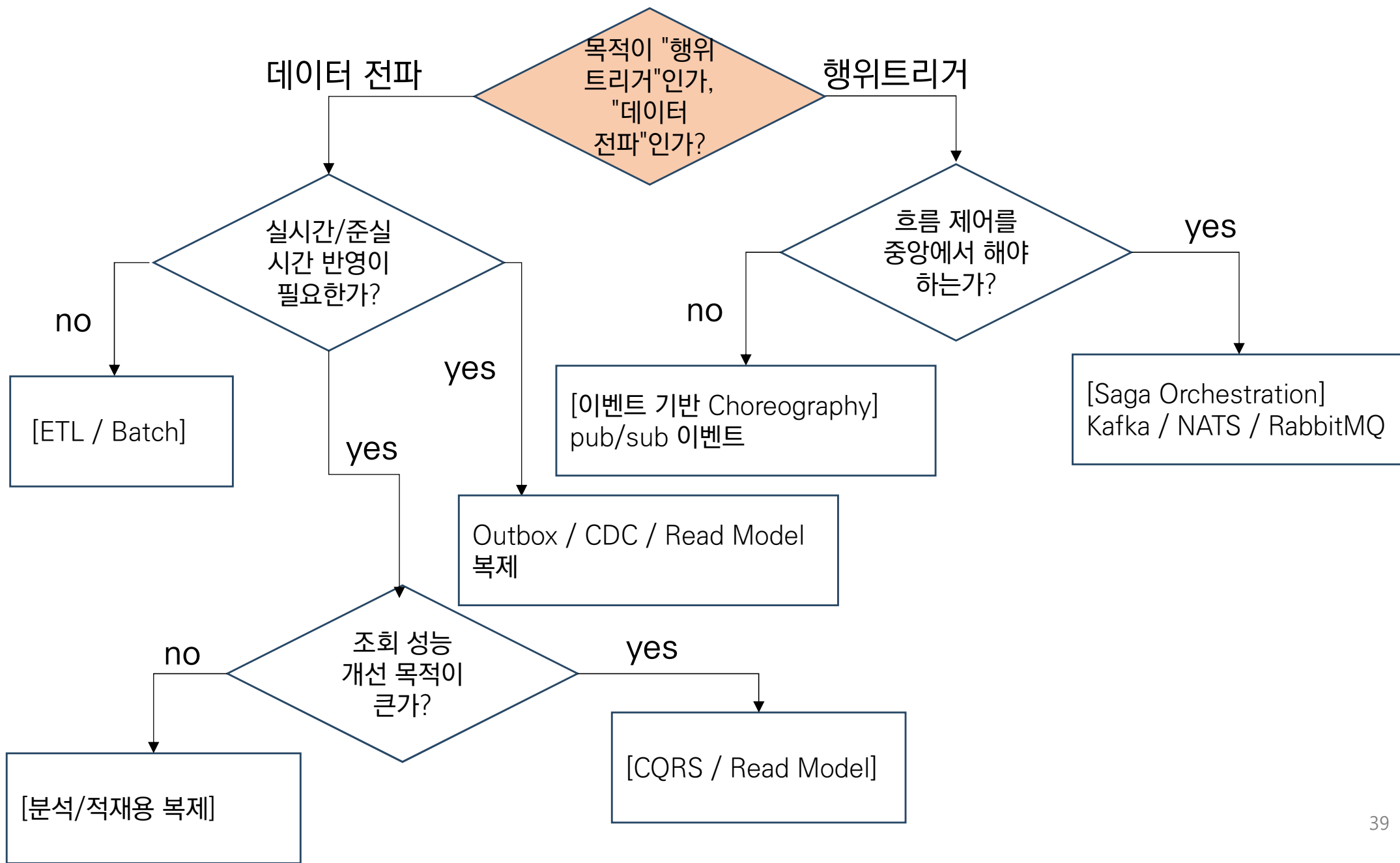
데이터 복제 안되는 경우

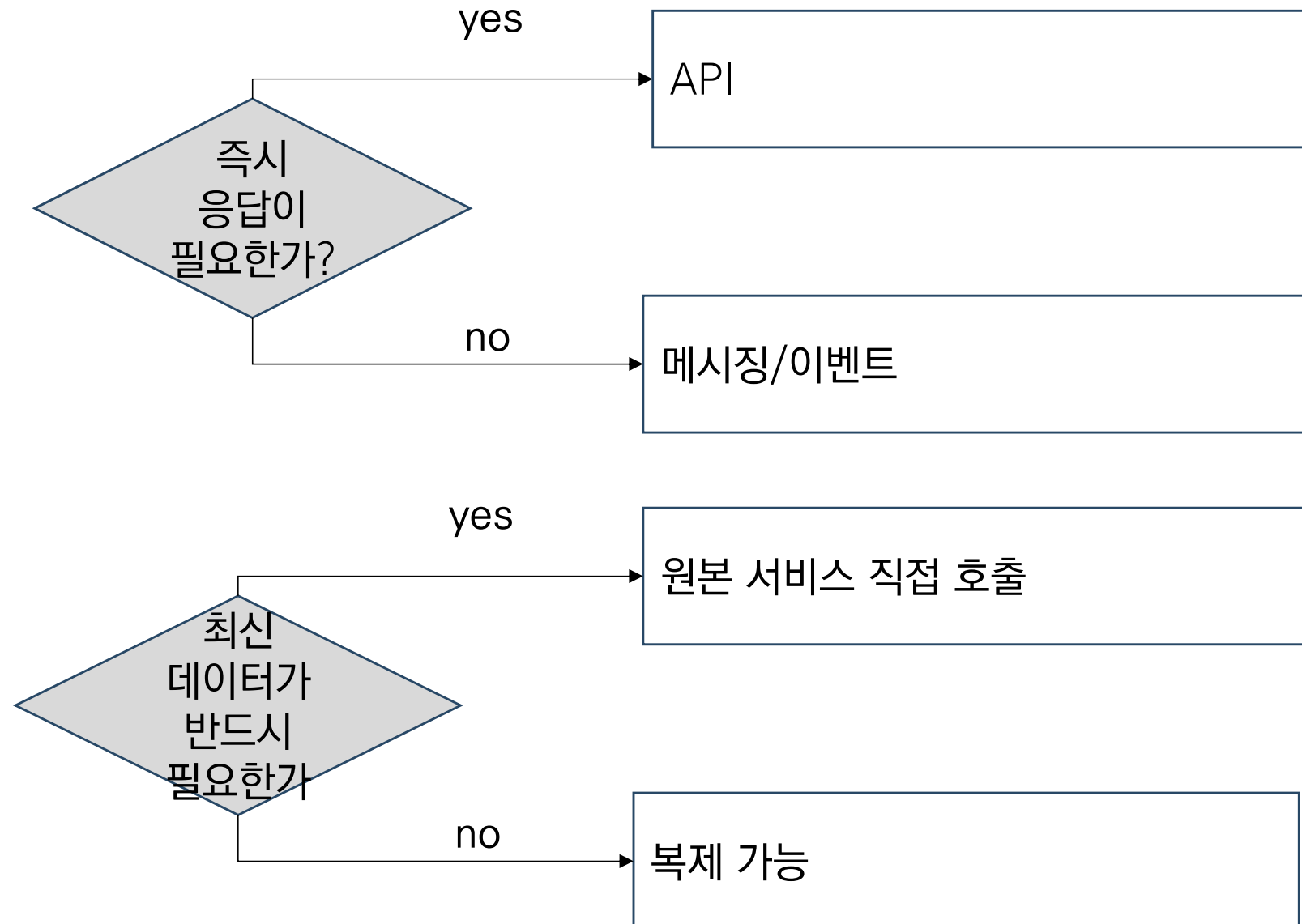
✓ 절대 복제하면 안 되는 경우

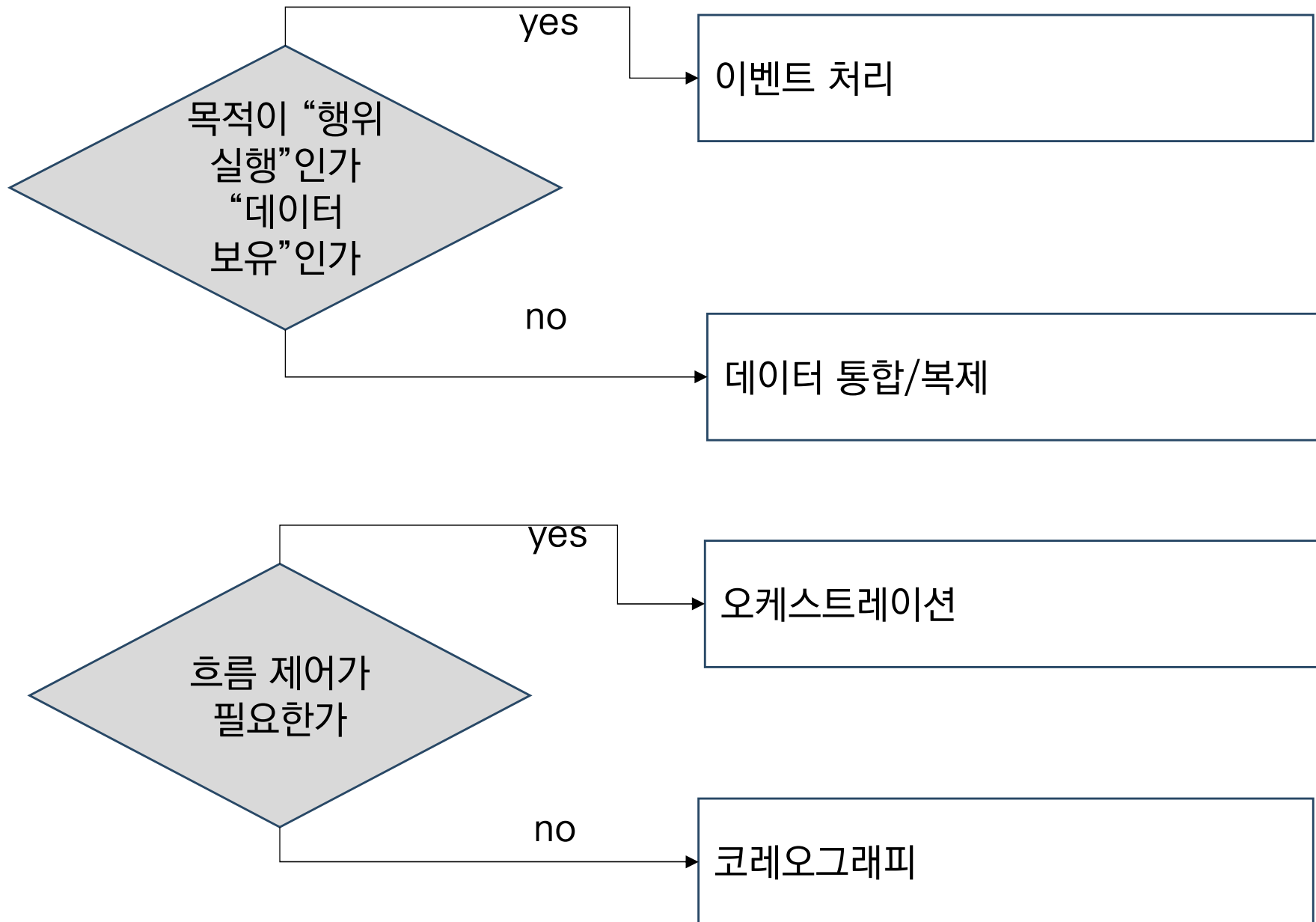
- 강한 일관성 필수
- 금융 잔액
- 결제 승인 상태

🔑 이걸 반드시 원본 서비스 호출

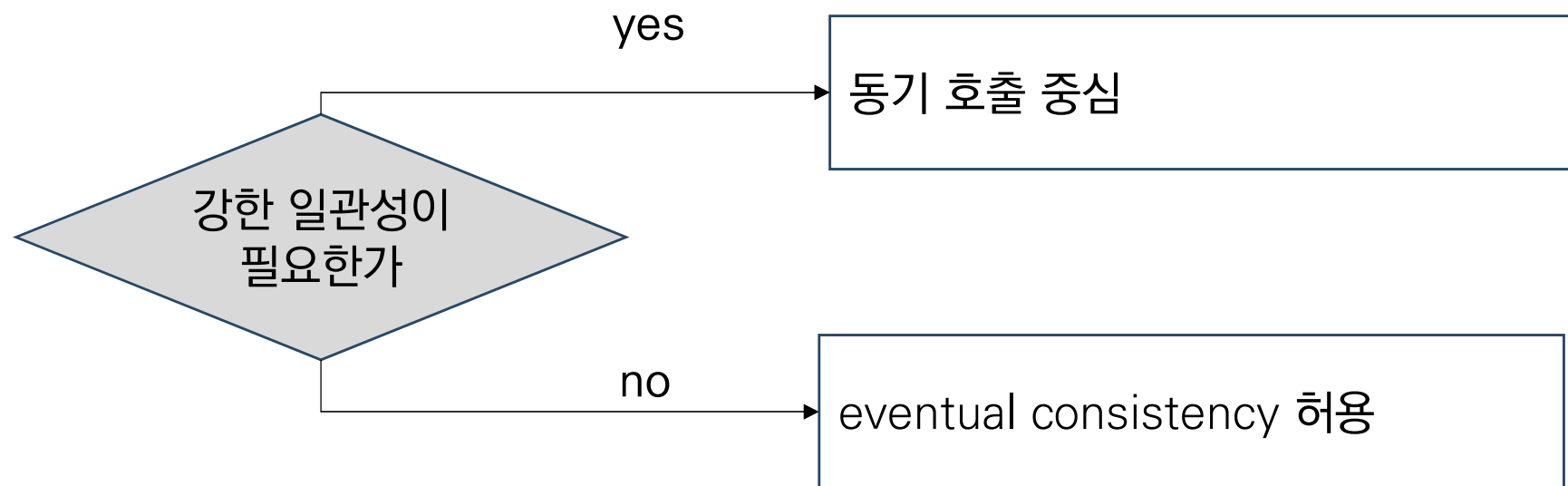








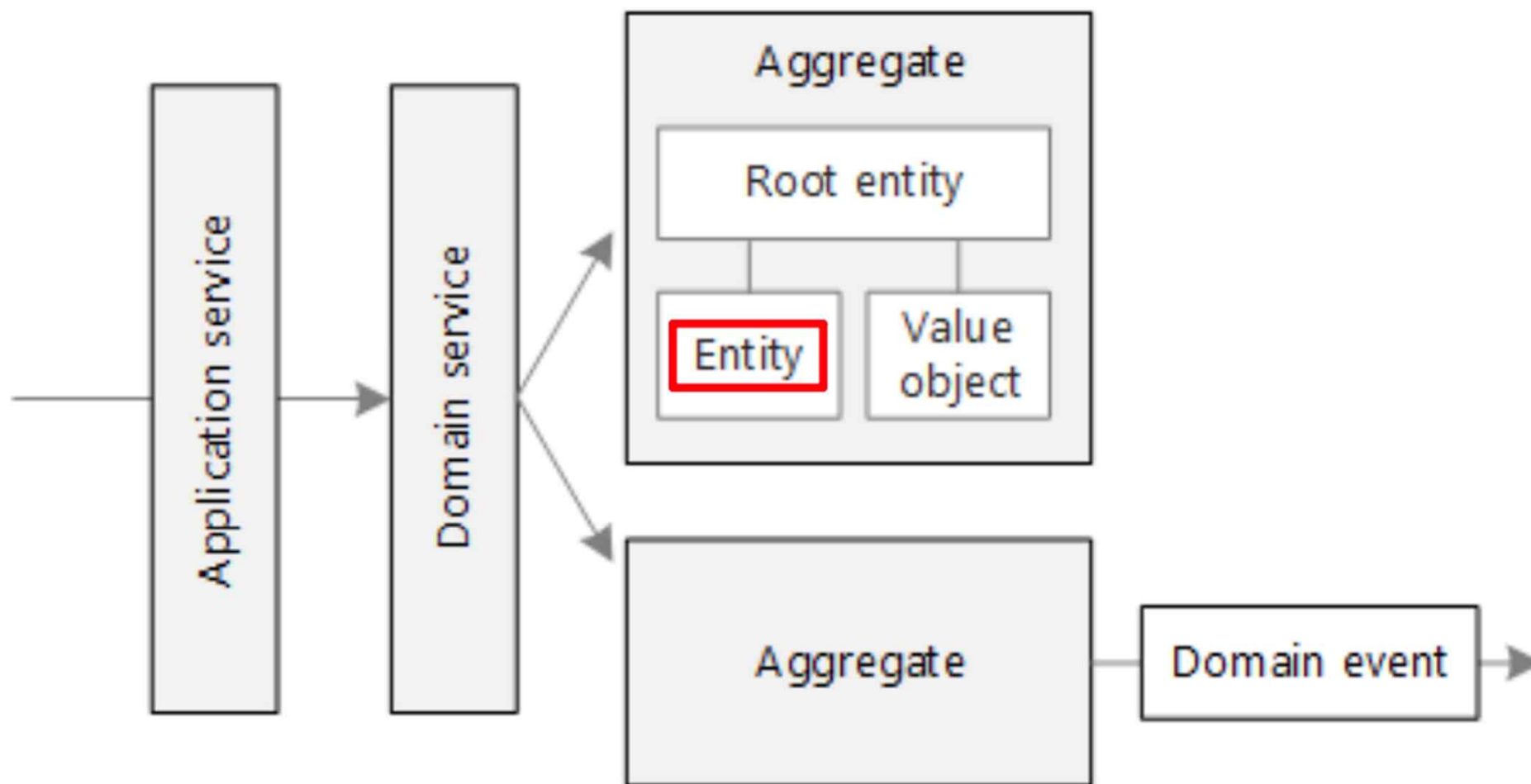
MSA 통신 방식 선택



판단 조건	선택 방식	대표 기술/패턴	적합한 상황
즉시 응답 필요	동기 API	REST, gRPC, Feign, WebClient	결제 승인, 로그인 검증, 최종 재고 확정
즉시 응답 필요 + 여러 서비스 조회 조합	API Composition	API Gateway, BFF	주문상세 화면, 통합 조회
후속 작업만 비동기로 넘기면 됨	비동기 메시징	Kafka, NATS, RabbitMQ	주문 생성 후 알림, 후처리
프로세스 순서를 중앙 통제해야 함	Saga Orchestration	Orchestrator + 메시지 브로커	주문→재고→결제→배송
서비스들이 자율적으로 반응하면 됨	이벤트 기반 Choreography	Pub/Sub 이벤트	알림, 로그, 후속 연계

판단 조건	선택 방식	대표 기술/패턴	적합한 상황
데이터 자체를 각 서비스가 가져야 함	데이터 통합/복제	Outbox, CDC, Read Model	조회 최적화, 독립성 확보
실시간성 낮고 분석 목적	배치 통합	ETL, Batch	DW, 리포트, AI 학습
강한 일관성 필수	직접 호출/원본 조회	동기 API, 단일 DB/단일 트랜잭션	결제, 정산, 계좌 잔액
약한 일관성 허용	이벤트/복제	Kafka, CDC, CQRS	대시보드, 분석, 재고 스냅샷

방식	장점	단점	선택 기준
동기 API	단순, 직관적, 즉시 결과 확인	강결합, 장애 전파	즉시 응답/최신 데이터 필요
비동기 메시징	느슨한 결합, 장애 격리	추적/디버깅 어려움	후속 처리, 비동기 허용
이벤트 처리	확장성 좋음, 서비스 자율성	흐름 가시성 낮음	후속 반응형 구조
Saga Orchestration	흐름 통제 쉬움	중앙 제어기 필요	주문/결제처럼 순서 중요
데이터 복제	조회 성능, 독립성, 분석 유리	일관성 지연	읽기 최적화, 통합 조회
ETL/Batch	대량 데이터 처리 유리	실시간성 낮음	리포트, AI, DW



Entity 설계

✓ 엔티티 개념 (정의)

- 엔티티 = 식별자(ID)를 기준으로 동일성이 유지되는 객체
- 즉: 값이 바뀌어도 같은 객체로 간주됨
- 시간에 따라 상태가 변화함
- 비즈니스적으로 “구별 가능한 대상”

✓ 예: 주문(Order)

- 상태: CREATED → PAID → SHIPPED
- 금액 변경 가능
- 주소 변경 가능

👉 그래도 같은 주문 → ID로 식별

Entity 설계

- ✓ 이 객체는 시간이 지나도 “같은 것”이라고 말할 필요가 있는가?
 - 예 → 엔티티
 - 아니오 → Value Object
- ✓ 설계시 주의사항
 - 상태 + 행위 중심으로 설계
 - DB 테이블 = 엔티티로 생각하지 말 것
 - 화면 기준 설계 금지
 - 과도한 엔티티 분리 금지
 - 다른 엔티티 직접 참조 최소화

도메인서비스 설계

✓ 도메인 서비스 개념

- DDD에서 가능한 한 로직은 Entity, 값 관련 로직은 Value Object
- 어떤 규칙은 여러 엔티티가 함께 관여하고, 특정 한 객체 책임으로 보기 어렵고, 비즈니스 규칙 자체는 매우 중요할 수 있음



도메인 서비스로 처리

도메인서비스 설계

✓ 예를 들어:

- 고객 등급 + 주문 금액 + 프로모션 정책을 종합해서 할인 계산
- 여러 창고 재고를 보고 어느 창고에 할당할지 결정
- 주문 가능 여부를 재고, 신용, 정책 기준으로 함께 판단
- 이건 단순 CRUD가 아니라 도메인 규칙

🔑 하나의 엔티티내에 넣기엔 이상함

도메인서비스 설계

- ✓ 여러 도메인 객체가 함께 관여
 - Order, Customer, Inventory, Policy 이들을 조합해 판단해야 할 때
- ✓ 특정 엔티티에 넣으면 책임이 부자연스러움
 - 예: Order가 창고 선택 전략까지 알 필요는 없음
- ✓ 기술 로직이 아니라 비즈니스 규칙 자체가 핵심
 - 예: 할인 규칙, 배정 규칙, 승인 규칙

도메인서비스 설계

- ✓ 기본 원칙
 - stateless
 - 도메인 용어 사용
 - 비즈니스 의미가 이름에 드러남

- ✓ 엔티티에 넣기 어색하면 도메인 서비스 검토
 - 예:여러 정책 조합
 - 여러 Aggregate 간 판단
 - 배정/선택/계산 전략

도메인서비스 설계 시 유의사항

- ✓ Application Service와 혼동하면 안됨
- ✓ Domain Service
 - 비즈니스 규칙
 - 도메인 판단
 - 도메인 용어 사용
- ✓ Application Service
 - 유스케이스 진행
 - 트랜잭션 관리
 - repository 호출
 - 외부 시스템 연동 orchestration

도메인서비스 설계 시 유의사항

- ✓ 단순 조회/CRUD를 도메인 서비스로 만들지 말 것
- ✓ 엔티티 책임을 빼앗지 말 것
- ✓ 기술 로직 넣지 말 것
 - 예:HTTP 호출, Kafka 발행, DB 저장, Redis 조회
- ✓ 지나치게 큰 “만능 서비스” 만들지 말 것
- ✓ 가능하면 무상태(stateless)로 유지
 - 도메인 서비스는 보통 상태를 갖지 않는게 자연스러움

Aggregate

- ✓ Aggregate 개념
 - Aggregate = 하나의 일관성(Consistency)을 보장하기 위한 객체 묶음
- ✓ Aggregate Root(루트 엔티티)를 통해서만 접근
- ✓ 트랜잭션 경계
 - 한 번의 트랜잭션 안에서 반드시 함께 일관성이 유지되어야 하는 범위
- ✓ 접근 제어 경계
 - 외부에서는 오직 Root만 접근 가능

Aggregate

- ✓ Aggregate 개념
 - Aggregate = 하나의 일관성(Consistency)을 보장하기 위한 객체 묶음
- ✓ Aggregate Root(루트 엔티티)를 통해서만 접근
- ✓ 트랜잭션 경계
 - 한 번의 트랜잭션 안에서 반드시 함께 일관성이 유지되어야 하는 범위
- ✓ 접근 제어 경계
 - 외부에서는 오직 Root만 접근 가능

```
Order (Root)
├── OrderLine
├── ShippingInfo
└── PaymentInfo
```

Aggregate 특징

특징	설명
Root 존재	반드시 루트 엔티티 있음
경계	내부 객체는 외부에서 직접 접근 금지
일관성	내부 규칙은 항상 유지
트랜잭션	하나의 트랜잭션 단위
참조 방식	다른 Aggregate는 ID로 참조

Aggregate 설계

- ✓ 무엇을 반드시 같이 유지해야 하는가라는 질문에서 시작됨
- ✓ 이 데이터들은 항상 같이 맞아야 하는가?
- ✓ 예:
 - 주문과 주문 항목 → 같이 유지 (Aggregate)
 - 주문과 결제 → 반드시 같이 유지할 필요 없음 (Aggregate 아님)

Aggregate 루트 선정

- ✓ Aggregate Root
 - 외부와 통신하는 창구
 - 내부 상태 변경 책임자
- ✓ 기준
 - 전체를 대표하는 객체
 - 식별자 보유
 - 외부에서 접근해야 하는 대상

👉 보통 Entity 중 하나가 Root가 됨

Aggregate 루트 선정

- ✓ 내부 객체 구성
 - Entity, Value Object
 - ↳ Root 내부에서만 관리
- ✓ 트랜잭션 범위 설정
 - 한 Aggregate = 한 트랜잭션
 - ↳ 이걸 기준으로 나눠야 함

Aggregate 선정 원칙

- ✓ Aggregate를 크게 잡지 말 것
 - 예: Order + Payment + Stock → 하나의 Aggregate
- ✓ 결과: 트랜잭션 길어짐
 - 락 증가
 - 성능 저하
 - MSA 의미 사라짐
- ✓ 작게 나누되 규칙은 반드시 지킬 것
- ✓ 너무 작으면:
 - 규칙 깨짐
 - 데이터 불일치

Aggregate 선정 원칙

- ✓ 다른 Aggregate는 직접 참조 금지
- ✓ 조회 요구사항으로 Aggregate 만들지 말 것
- ✓ 하나의 Aggregate는 하나의 Repository
- ✓ Aggregate 간 트랜잭션은 이벤트로 해결

Repository

- ✓ Repository = Aggregate를 저장하고 조회하는 인터페이스 (도메인 관점의 저장소)
- ✓ 역할
 - Aggregate 저장(save)
 - Aggregate 조회(find)
 - 도메인 모델을 영속성(DB)으로부터 분리

Repository 설계 유의사항

- ✓ Aggregate Root 기준
 - OrderRepository $\Rightarrow$ OK
 - OrderLineRepository $\Rightarrow$ NOT OK
- ✓ 비즈니스 로직 없음
- ✓ 저장/조회만 담당
- ✓ 정책/규칙은 Entity나 Domain Service
- ✓ 유의 사항
 - DAO처럼 쪼개지 말 것
 - 화면 조회용 쿼리 남발 금지 $\rightarrow$ CQRS로 분리
 - 다른 Aggregate 직접 조회 남용 금지

도메인 이벤트

- ✓ 개념: 도메인 이벤트 = 도메인에서 “이미 발생한 사실”을 표현하는 객체
- ✓ 역할
 - 서비스 간 비동기 협력
 - 후속 처리 트리거
 - 데이터 전파 (복제/분석)
- ✓ 특징
 - 과거 사실
 - 불변
 - 비동기 전달

도메인 이벤트 유의사항

- ✓ DB 구조 그대로 보내지 말 것
- ✓ 필요한 데이터만 포함
- ✓ 중복 처리 고려 (idempotent)
- ✓ 이벤트 스키마 변경 대비 (버전 관리)

Q & A